GSA Gateway

A Local-First, Database-Centric Architecture for Grounded Multi-Platform Conversational Assistance

Mohammad Dindoost
NJIT Graduate Student Association

Technical Report — June 2026

Abstract

GSA Gateway is a privacy-preserving, locally hosted conversational assistant and administrative platform serving a university graduate student association across Discord and Telegram. The system answers free-form questions about events, funding, policy, and resources using a fully local Retrieval-Augmented Generation (RAG) pipeline grounded in a curated knowledge base, with no data leaving the host. This report presents the system's technical architecture and the engineering decisions behind it. We describe the evolution from a conventional vector-store design to a unified single-file architecture in which dense vector search (`sqlite-vec`), lexical full-text search (FTS5), relational data, and configuration all live in one SQLite database; dense and sparse rankings are fused with Reciprocal Rank Fusion. We present a platform-agnostic *connector* model that fans a single logical message out to multiple channels with a delivery audit trail, a database-centric data model (organization hierarchy, versioned knowledge items, posts as a universal output type, inherited settings), and a methodology for migrating a live production system through feature-flagged coexistence with instant rollback. We close with a reliability case study, a qualitative evaluation, and limitations. The recurring theme is a deliberate bias toward *locality, auditability, and graceful degradation* over scale.

1 Introduction

Student organizations accumulate institutional knowledge—deadlines, funding rules, contacts, event logistics—that is chronically hard for members to retrieve. GSA Gateway addresses this by exposing that knowledge through natural-language chat on the platforms students already use, while remaining cheap to operate and respectful of privacy. The design is shaped by four hard requirements:

1. **Grounded answers.** The assistant must answer *only* from a curated knowledge base and decline otherwise; fabricated answers are unacceptable in an official channel.
2. **Privacy and locality.** No student data or query content may leave the host. This precludes hosted LLM APIs and external vector databases.
3. **Multi-platform reach.** The same assistant and the same broadcasts must serve Discord and Telegram, with room for future channels.
4. **Low operational burden.** The system runs on a single always-on machine and is administered by a non-specialist officer.

These constraints drive most of what follows. The technical contributions of the system are: (i) a fully local RAG stack (embeddings and generation via Ollama) with principled grounding and degradation; (ii) a *single-database* hybrid retrieval engine that replaces a separate vector store by

combining `sqlite-vec` KNN and FTS5 BM25 under Reciprocal Rank Fusion; (iii) a connector abstraction decoupling message content from platform-specific delivery, with a persistent delivery log; (iv) a database-centric, multi-tenant data model with versioned knowledge and inherited settings; and (v) a coexistence strategy that let a second-generation architecture be introduced into a running system behind feature flags, with drop-in interface adapters and instant rollback.

2 System Architecture

GSA Gateway runs as a single bot process (with a companion Telegram poller) on one host. Architecturally it is layered:

- **Interaction layer.** Slash commands and an `on_message` handler for free-form chat; an intent detector routes greetings, thanks, and trivial queries away from the expensive RAG path.
- **Retrieval layer.** Converts a question into grounding context (the focus of §3 and §4).
- **Generation layer.** A local LLM produces an answer constrained to the retrieved context, or the system degrades to lexical search.
- **Data layer.** A SQLite database holds operational records and, in the second-generation design, the entire knowledge and configuration substrate.
- **Delivery layer.** A connector registry transmits outbound messages to every targeted platform (§6).

A defining characteristic is that the system exists in two coexisting generations. The first (`v1`) is the deployed application built around a dedicated vector database (ChromaDB). The second (`v2`) is a database-first redesign that consolidates storage and adds hybrid retrieval; it is wired *into* the running v1 process behind feature flags rather than deployed as a separate service. §8 explains why this coexistence was the safest migration path.

3 Grounded Retrieval-Augmented Generation

3.1 Offline indexing

Knowledge-base documents are split into chunks of at most ≈ 350 tokens at semantic boundaries, balancing two competing pressures: chunks must be small enough that a retrieved unit is densely relevant, yet large enough to carry self-contained meaning. Each chunk is embedded with `nomic-embed-text` (768 dimensions) served locally by Ollama. The model is *asymmetric*: documents are embedded with a `search_document:` prefix and queries with `search_query:`. Using the wrong prefix silently degrades recall, so the distinction is enforced at the API boundary. Embeddings are L_2-normalized at write time so that the default Euclidean index ranks identically to cosine similarity:

$$\|a - b\|_2^2 = 2\,(1 - \cos(a, b)) \quad \text{for } \|a\| = \|b\| = 1.$$

3.2 Online answering

A query is embedded, matched against the index by approximate nearest neighbor, reranked, and the top chunks become grounding context for the LLM (`llama3.1:8b`). The system prompt instructs the model to answer strictly from the provided context and to refuse when the answer is absent—the mechanism that keeps an official channel from emitting confident falsehoods. Per-user conversation

memory (a short, time-bounded window) supports follow-up questions without unbounded context growth.

3.3 Graceful degradation

If the LLM is unavailable, the assistant does not fail: it falls back to fuzzy lexical matching (`rapidfuzz`) over the knowledge base, returning the best matching entry below a confidence threshold rather than a generated answer. This makes the heavy generation component optional—valuable on a resource-constrained host, where it can be disabled deliberately to free memory.

4 Hybrid Retrieval in a Single Database

4.1 Motivation: collapsing the storage tier

The first-generation design used three stores: ChromaDB for vectors, SQLite for records, and in-memory fuzzy search. This works but multiplies operational surface: separate persistence, separate backup semantics, and no transactional consistency between an item and its embedding. The second generation collapses all of it into one SQLite file using two extensions: `sqlite-vec` provides a `vec0` virtual table for native KNN over `FLOAT[768]` vectors, and FTS5 provides BM25-ranked full-text search. Triggers keep the FTS index synchronized with the canonical `knowledge_items` table. The payoff is a single portable artifact with atomic writes, one backup, and no cross-store drift.

4.2 Fusing dense and sparse signals

Dense (semantic) retrieval captures paraphrase and conceptual similarity but can miss exact tokens; sparse (lexical) retrieval excels at names, acronyms, and rare terms but misses synonymy. The system runs both and fuses their *rankings* with Reciprocal Rank Fusion (RRF):

$$\mathrm{score}(d) \;=\; \sum_{i \in \{\mathrm{dense},\,\mathrm{sparse}\}} \frac{1}{k + \mathrm{rank}_i(d)}, \qquad k = 60.$$

RRF combines result lists by rank rather than by raw score, which sidesteps the incompatibility between cosine similarities and BM25 magnitudes and is robust without per-query tuning. A small type-dependent boost is then applied (e.g. favoring `contact` and `event` items) to reflect that certain question classes have canonical answer types.

4.3 A production lesson: decoupling the candidate pool

An early failure illustrates a subtle retrieval pitfall. The candidate pool fed to fusion was initially sized as a multiple of the requested result count (pool $= \max(c \cdot \mathrm{limit}, 15)$). For a query whose correct answer ranked highly in the *sparse* list but only moderately in the *dense* list, a small pool truncated the answer out of the dense candidates before fusion could rescue it, and a less relevant item was returned. The fix was to decouple the pool from the result count—using a fixed, generous candidate pool (e.g. 40) independent of how many results the caller ultimately wants. The general principle: *fusion quality is bounded by candidate recall*, so the pool must be sized for recall, not for the final cut.

5 Data Model

The second-generation schema is deliberately database-centric: structure, content, and configuration are all rows, not code or files.

- **Organization hierarchy.** An `organizations` table forms a tree via `parent_id` (university → college → department → club, etc.), traversed with recursive common table expressions. This makes the platform multi-tenant: a single deployment can host many sub-organizations.
- **Versioned knowledge.** `knowledge_items` are typed and versioned: a `root_id` groups all versions of an item, `parent_id` links to the prior version, and `is_active` marks the current one. Edits append a new version rather than mutating in place, preserving history and enabling restore.
- **Posts as universal output.** Every outbound message—one-off, recurring template, or event announcement—is a row in a single `posts` model, decoupling *what* is said from *where* it is delivered.
- **Inherited settings.** A `settings` table stores configuration per organization; lookups walk the ancestor chain (self → parent → root), so a value set at the root is inherited by all descendants unless overridden. The same mechanism stores controlled *vocabularies* (valid organization types, knowledge types, contact roles), letting each deployment define its taxonomy as data.

Tables use SQLite's `STRICT` mode for type enforcement, and all timestamps are stored as canonical UTC and rendered in the organization's local timezone at the edge, avoiding a class of off-by-hours scheduling bugs.

6 Multi-Platform Delivery: the Connector Pattern

Outbound messaging is built around a single abstraction. A `Post` describes *what* to send (content, optional signature, target platforms and channels). A `BaseConnector` subclass knows how to send for *one* platform (`send_text`, `send_media`, `send_interactive`). A `ConnectorRegistry` fans a post out to all targeted connectors in parallel and— critically—*never raises*: each delivery is wrapped, failures become recorded failure results, and every attempt is written to a `post_deliveries` audit table.

This yields three properties. First, **extensibility**: adding a platform is one new subclass and a registration, with no change to callers. Second, **isolation**: a misbehaving platform cannot sink a multi-channel broadcast. Third, **auditability**: `post_deliveries` is a durable record of what was sent where, the substrate for a "single source of truth" delivery policy. A scheduler materializes due recurring templates and event reminders into concrete posts and hands them to the registry, all on UTC time.

7 Case Study: Fault-Tolerant Live Event Tracking

A live sports tracker exercises the system differently from Q&A: it polls an external API on a fixed cadence, diffs successive snapshots of match state to emit *events* (kickoff, goal, half-time, full-time), deduplicates via per-match state, and publishes each event through the connector registry. Two reliability lessons emerged.

First, a long-lived HTTP client session that was reused across polls suffered a 100% failure rate in production while a one-shot request to the same endpoint succeeded: the pooled TCP connection went stale between polls and every reuse was reset. The remedy was to use a fresh client session per poll with bounded retries. Second, the upstream API itself proved intermittently unavailable

(roughly one connection in six was dropped even for fresh sessions). Because the tracker polls on a short cadence and each event need only be reported once within a small window, retries combined with the poll interval yield effectively complete coverage despite a flaky dependency. The takeaway generalizes: for periodic integrations, prefer *stateless* requests and design for an unreliable upstream rather than assuming a healthy long-lived connection.

8 Engineering Methodology

8.1 Feature-flagged coexistence and instant rollback

The most consequential decision was *not* to deploy the second generation as a replacement. Instead, v2 components are imported into the running v1 process and gated by environment flags. A drop-in adapter (a "shim") presents the exact interface the v1 code expects, so the new hybrid retriever can substitute for the old vector-store retriever transparently. Any flag can be flipped off and the process restarted to return to v1 behavior in seconds. This converts a high-risk rewrite into a sequence of independently reversible steps—essential when the system is a live service with a fixed external deadline.

8.2 Safe data migration

Schema and data migrations are idempotent and guarded: they run first against a copy of the live database (a dry run), and any live run is preceded by an automatic, timestamped backup. Administrative writes from the dashboard are likewise idempotent (`INSERT OR REPLACE`), so re-applying a change is harmless.

8.3 A dual-mode administrative interface

Administration is intentionally dependency-free. The dashboard is static HTML/CSS/JavaScript that reads a SQLite database in the browser via WebAssembly (`sql.js`). It operates in two modes. In *file mode* it reads a database copy and emits changes as SQL patch scripts to be applied out of band—never writing the live file directly. In *server mode* a tiny standard-library HTTP server on the host (bound to localhost only, reached over an SSH tunnel) serves the dashboard and mediates reads and writes against the live database, returning WAL-consistent snapshots. The same UI thus supports both a zero-trust "inspect and export" workflow and a fully interactive one, without shipping a heavyweight web framework.

8.4 Single-source-of-truth governance

Because any process holding platform credentials can post directly, "only allow posts that originate in the database" cannot be enforced at the platform boundary. The practical model is constructive: route all sends through the connector registry (which logs them), keep credentials in a single process, retire ad-hoc senders, and reconcile channel activity against `post_deliveries` to detect anything out of band.

9 Evaluation

Evaluation to date is qualitative and operational rather than a formal benchmark, which we state plainly as a limitation. Retrieval behavior was validated against a suite of representative queries

spanning funding, contacts, and cross-domain questions; the hybrid-with-fixed-pool design corrected concrete failures of the earlier vector-only retriever (notably surfacing the correct office for finance questions). Correctness of the non-retrieval machinery—schema, hybrid ranking, connectors, scheduling, the administrative server, and event detection/deduplication—is covered by an automated unit-test suite, alongside the inherited first-generation tests. Operationally, the system has run as an always-on service, and the connector audit trail provides end-to-end confirmation of multi-platform delivery.

10 Limitations and Future Work

The deployment is single-host and vertically scaled; horizontal scale is out of scope and arguably unnecessary at this size. Answer quality and latency are bounded by a local 8-billion-parameter model. Retrieval is validated qualitatively; a labeled relevance benchmark would enable quantitative tuning of the fusion constant and type boosts. The taxonomy vocabularies guide but do not constrain values at the database level. Finally, the two generations still coexist: completing the migration entails re-implementing the remaining interaction surface (the full command set and ancillary schedulers) on the v2 substrate so the first generation can be retired, and unifying rich, platform-specific message formatting within the connector model.

11 Conclusion

GSA Gateway shows that a small, local-first system can deliver grounded, multi-platform conversational assistance without external dependencies. Its more transferable ideas are architectural: consolidating dense vectors, lexical search, and relational data into a single SQLite file with rank-fusion retrieval; separating message content from platform delivery behind an auditable connector registry; and migrating a live system safely through feature-flagged coexistence with drop-in adapters and instant rollback. Throughout, the design trades scale for locality, auditability, and graceful degradation—an appropriate and, we argue, principled choice for community-scale software.